



ProjectX Platform

DEEP-DIVE TECHNICAL ARCHITECTURE & OPERATIONAL PLAYBOOK

A comprehensive technical manual detailing the underlying architecture, cryptographic security model, backend topology, multi-threaded UI lifecycle, advanced routing mechanics, and the step-by-step user journey governing the next generation of zero-trust Operational Technology (OT) network access.

Confidential & Proprietary Document
Generated: June 2026

1. Executive Summary & Design Philosophy

ProjectX is an enterprise-grade Zero-Trust network access platform engineered specifically to provide secure, role-based remote access to Industrial and Operational Technology (OT) devices. Traditional VPN solutions rely on granting sweeping subnet access to operators, posing massive lateral-movement risks.

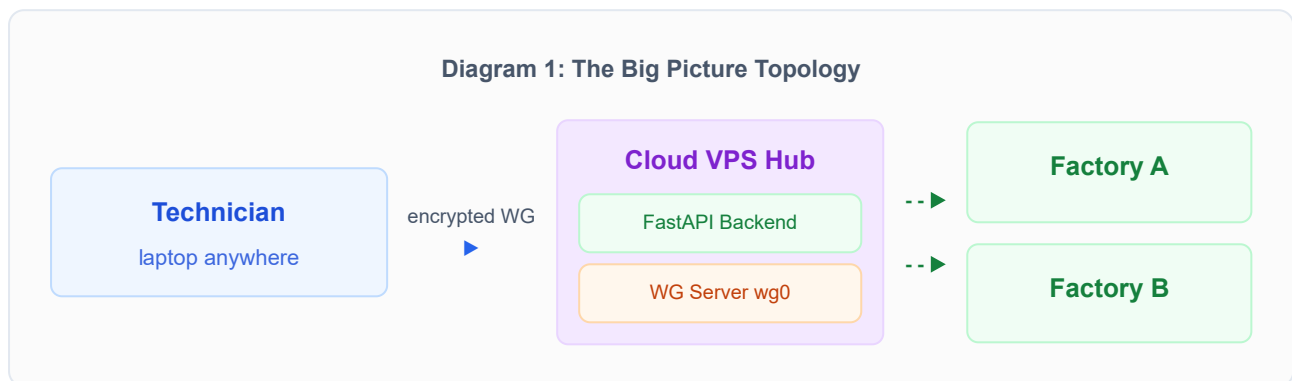
By fusing the blisteringly fast cryptography of WireGuard with a robust, centrally orchestrated FastAPI backend, ProjectX completely abstracts away the complexity of managing private keys and iptables. It enforces a strictly governed Hub-and-Spoke topology where every connection is authenticated via short-lived JWTs, every packet is explicitly routed via dynamic NAT, and every user action is validated against their cryptographic identity.

1.1 The Complete Technology Stack

Domain	Implementation & Technologies
Backend Orchestrator	Python 3.13 utilizing FastAPI for asynchronous REST APIs. Uvicorn acts as the ASGI server. Data is modeled via Pydantic and persisted via SQLAlchemy to an encrypted SQLite database for ultra-low latency reads.
Cryptography	Strictly relies on Curve25519 for Diffie-Hellman key exchange, ChaCha20 for stream encryption, and Poly1305 for MAC authentication, leveraging native WireGuard kernel drivers.
Gateway Routing Hub	A native Windows Server executing <code>wireguard.exe</code> as a system service. It utilizes custom Kernel-level <code>WeakHostSend</code> registry edits to achieve flawless peer-to-peer U-turn routing across isolated virtual subnets.
Desktop Endpoints	A highly optimized C++ wrapped GUI utilizing PyQt6. It leverages complex multi-threading (<code>QThread</code>) to execute LAN sweeps, websocket connections, and cryptographic parsing entirely asynchronously.
Real-Time Synchronization	WebSockets (<code>ws://</code> / <code>wss://</code>) establish a persistent bi-directional channel, broadcasting JSON payloads to connected clients within milliseconds of a state change in the FastAPI controller.

2. Network Architecture & Hub-and-Spoke Topology

Unlike mesh networks where endpoints must discover one another, ProjectX enforces a centralized Hub-and-Spoke topology. Industrial gateways (Factory nodes) and remote operators (Technician laptops) never establish direct internet connections with each other. Instead, both nodes establish an outbound-only connection to the Cloud Gateway.



2.1 Bypassing the Windows Strong Host Model (Hairpin Routing)

A significant engineering hurdle in Hub-and-Spoke VPNs on Windows Servers is the OS's native "Strong Host Model". By default, the Windows TCP/IP stack silently drops any packet that attempts to route back out the exact same interface it arrived on. This makes Peer-to-Peer ping through a central `wg0` interface impossible natively.

The ProjectX Backend API mitigates this programmatically. When generating the WireGuard tunnel configuration, the Python controller invokes native PowerShell to actively manipulate the Windows IP routing stack, enabling `WeakHostSend` and `WeakHostReceive` on the virtual adapter. This allows the packet to perform a "U-Turn" back to the destination peer.

```
# Backend implementation of Hairpin enablement
subprocess.run([
    "powershell", "-Command",
    "Set-NetIPInterface -InterfaceAlias 'wg0' "
    "-Forwarding Enabled -WeakHostSend Enabled -WeakHostReceive Enabled"
], check=True)
```

3. Identity, Roles, and Code of Conduct

Access in ProjectX is governed by a strict hierarchical system. Every user account belongs to a designated `tenant_id`. A fundamental design principle of ProjectX is **absolute tenant isolation**. Below is the strict Code of Conduct and capability matrix for each identity.

3.1 Role Hierarchy & Responsibilities

Role	Tenant Bounds	Code of Conduct & Responsibilities
System Owner	Global	The Architect. Operates outside the tenant structure. Responsible solely for platform creation, generating new Tenants, and issuing 16-character Activation Keys to new Corporate clients. Must never interact with ground-level factory endpoints.
Master	Isolated	The Corporate IT Admin. Claims their Tenant using an Activation Key. Responsible for setting up the internal corporate structure, adding sub-users (Admins/Trusted), and generating the cryptographic configurations that are deployed onto the physical Factory Gateways. They control Cross-Tenant sharing.
Second Master	Isolated	The Backup Admin. Granted identical rights to the Master to prevent catastrophic lockouts. Responsible for auditing network access if the primary Master is unavailable.
Admin	Isolated	The Standard Operator. A restricted user. They can only see and connect to devices that have been <i>explicitly</i> assigned to them by a Master. Perfect for third-party contractors or localized engineers.
Trusted	Isolated	The Elevated Operator. A highly privileged user. They inherently have full access to <i>all approved devices</i> across the entire Tenant without needing explicit, per-machine assignments. Ideal for Senior Field Engineers.

3.2 Mandatory 2FA & TOTP

All administrative operations require Time-based One-Time Passwords (TOTP). During onboarding, the FastAPI backend generates a secret seed using the `pyotp` library, transmitting a `otpauth://` URI to

the PyQt client, which dynamically renders a QR Code in memory for scanning by Google Authenticator or Authy.

4. The Operational Playbook (User Journey)

The following maps out the exact sequence of events required to bring a brand-new Corporate Client online and establish a secure connection from a Technician's laptop to an Industrial PLC.

Phase 1: Inception (System Owner)

1. The System Owner logs into the global dashboard using highly restricted credentials.
2. They create a new Tenant (e.g., "MegaCorp Industries").
3. The system generates a unique, one-time **16-character Activation Key**.
4. The Owner securely transmits this Activation Key to MegaCorp's IT Director.

Phase 2: Tenant Claiming (Master)

1. MegaCorp's IT Director downloads the PyQt Desktop Client.
2. They click "*Activate Master Account with Key*" on the login screen.
3. They input the Activation Key, their corporate email, and a password.
4. They are immediately prompted to bind an Authenticator App (Google Auth) via an on-screen QR Code.
5. Once 2FA is verified, the Tenant is permanently locked to their identity. They are now the **Master**.

Phase 3: Provisioning Gateway Devices & Users

1. The Master logs into the Web Dashboard.
2. They create a "Gateway Endpoint" device profile for *Factory A*. The backend generates a WireGuard `.conf` file and an Installation Script.
3. They deploy the script onto the physical Windows/Linux server sitting inside Factory A. The server boots, connects to the Cloud Hub, and begins emitting a 30-second telemetry heartbeat.
4. The Master creates an **Admin** user account for their remote Field Technician (John).

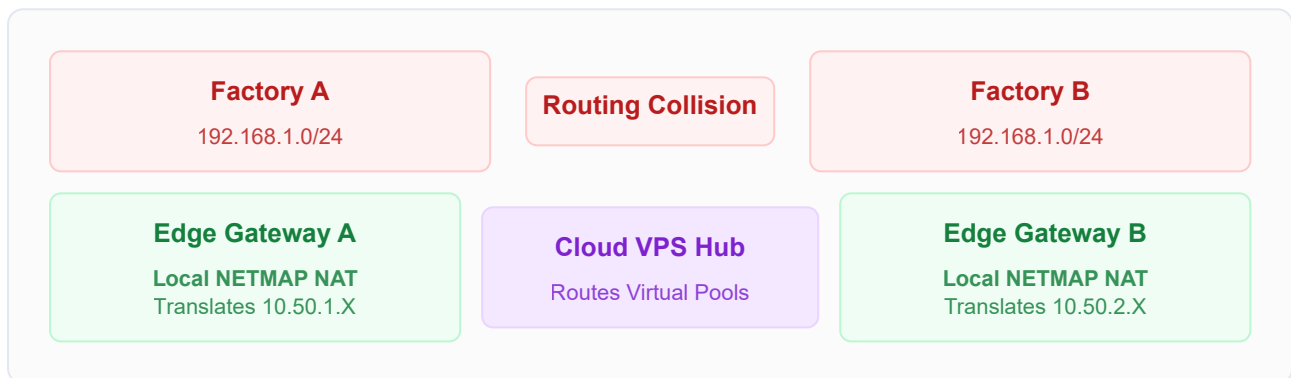
Phase 4: Remote Access Execution (Admin)

1. John (the Technician) opens the PyQt Desktop App from his hotel room.
2. He logs in and sees *Factory A* listed in his device grid with a green online heartbeat indicator.
3. He clicks the "**Connect**" toggle.
4. The PyQt App parses his cryptographic identity, builds an encrypted WireGuard tunnel to the Cloud Hub, and the FastAPI backend calculates the U-Turn routing metric.

5. John opens his browser, types in the internal IP of the Factory PLC (e.g., `10.50.1.15`), and the interface loads instantly. The encrypted tunnel is fully operational.

5. Subnet Collisions and Edge NAT Translation

In industrial environments, it is incredibly common for disparate factories to use identical, overlapping internal subnets (e.g., `192.168.1.0/24`). Normally, this causes catastrophic routing collisions. Because the central Hub runs on Windows Server, attempting to centrally route and NAT overlapping subnets without VRFs (Virtual Routing and Forwarding) is impossible.



ProjectX solves this implicitly via **Dynamic Edge NAT**. When the FastAPI backend detects a subnet collision, it assigns a unique Virtual Pool (e.g., `10.50.1.0/24`) to the Factory. It automatically instructs the Central Hub to route that virtual subnet down the tunnel.

Crucially, the translation happens at the absolute Edge. The backend automatically injects native Linux `iptables NETMAP` scripts directly into the 'PostUp' block of the Factory Gateway's WireGuard configuration file. When a Technician connects to `10.50.1.15`, the packet routes to the Factory Gateway, which instantly translates the destination IP to `192.168.1.15` before forwarding it to the PLC.

6. Desktop Client Engineering & UI Synchronization

The desktop client utilizes a sophisticated C++ wrapped GUI via PyQt6. It is designed to be completely non-blocking, relying heavily on the PyQt `QThread` and Signal/Slot architecture.

6.1 Worker Lifecycle Management

Because network requests (like initiating a WireGuard tunnel or verifying 2FA) take time, executing them on the main GUI thread would cause the application to freeze ("Not Responding"). ProjectX offloads all I/O bound tasks to isolated `QThread` workers (e.g. `ConnectWorker` , `ScanWorker`).

To prevent PyQt's aggressive garbage collector from destroying running threads when a user abruptly closes the application (which historically causes the fatal `QThread: Destroyed while thread is still running` crash), the application utilizes a global `_active_workers` tracking set. During the application's `closeEvent` , the thread manager systematically calls `terminate()` and `wait()` on every background process, ensuring a perfectly clean exit.

6.2 Multithreaded LAN Discovery

When installed on an industrial gateway, operators must identify which PLCs and sensors are accessible. The PyQt Client features a high-performance LAN Scanner. It spawns an internal thread pool with up to 50 concurrent workers executing parallel ICMP Ping Sweeps against the local subnet. It then parses the host OS's ARP table via Regex to extract MAC addresses and maps hostnames, syncing the physical topography instantly to the cloud.

6.3 Real-Time WebSocket Synchronization & Offline Resilience

A persistent WebSocket connection binds every client application to the FastAPI server. A background `WebSocketWorker` runs a blocking `run_forever()` loop listening for JSON payloads. When an endpoint comes online, is renamed, or is deleted by an admin anywhere in the world, the backend triggers `ws_manager.broadcast()` . The client receives this payload and emits a PyQt Signal to the main thread, instantaneously triggering UI repaints without requiring manual API polling.

Because industrial internet connections drop frequently, the ProjectX client caches all vital configurations inside an encrypted local SQLite database. If the internet fails, the UI falls back to this localized state, preventing crashes. When connectivity is restored, queued status changes are pushed upstream seamlessly.